

Resource Management

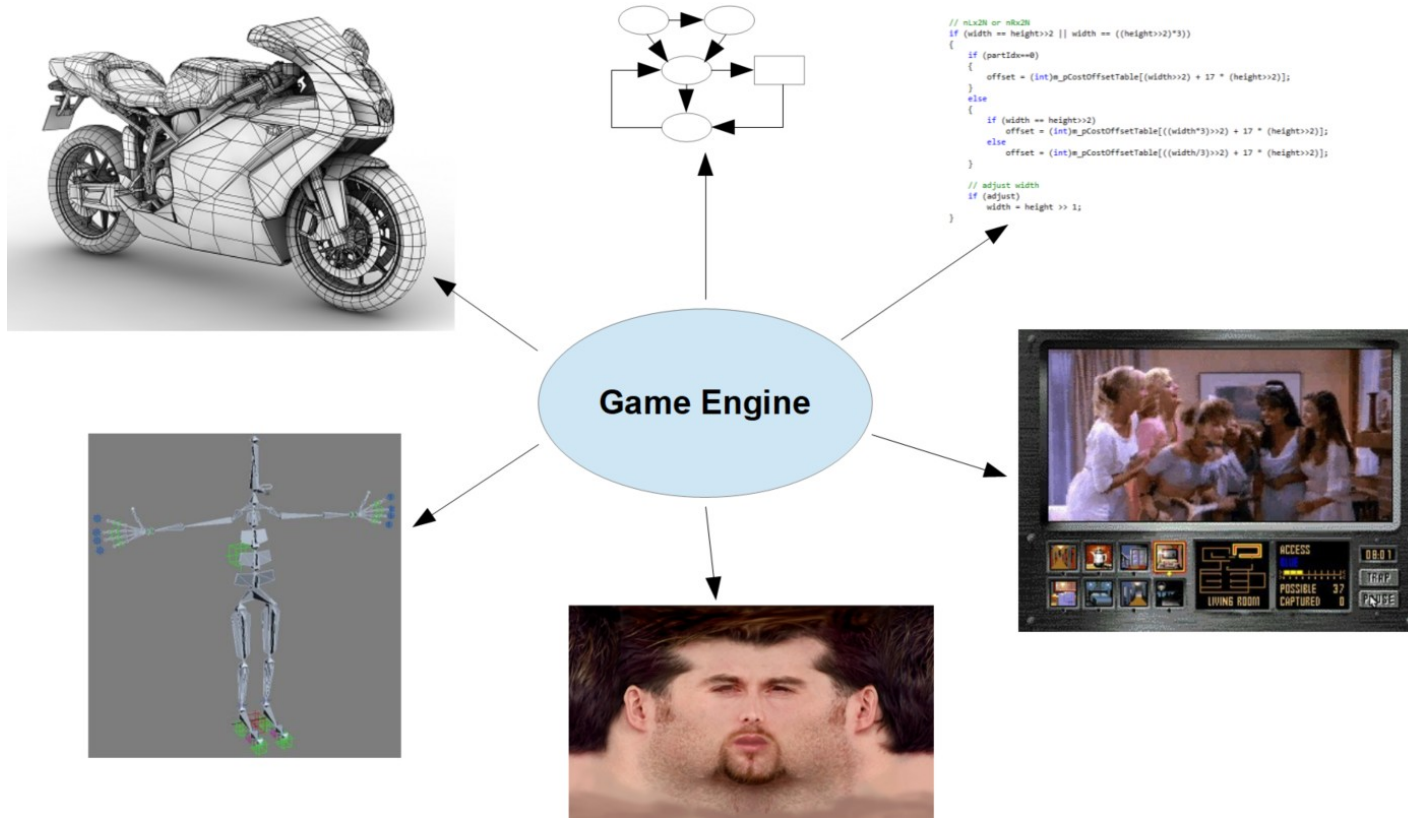


Image sources: <http://arsavar.com/wireframe-model#>, <http://maulik13.blogspot.de/2006/10/my-first-3d-character-rig.html>, http://images2.wikia.nocookie.net/half-life/en/images/a/ab/Trav_facemap.png, <http://tvtropes.org/pmwiki/pmwiki.php/Main/FullMotionVideo>

Resource Management - Overview

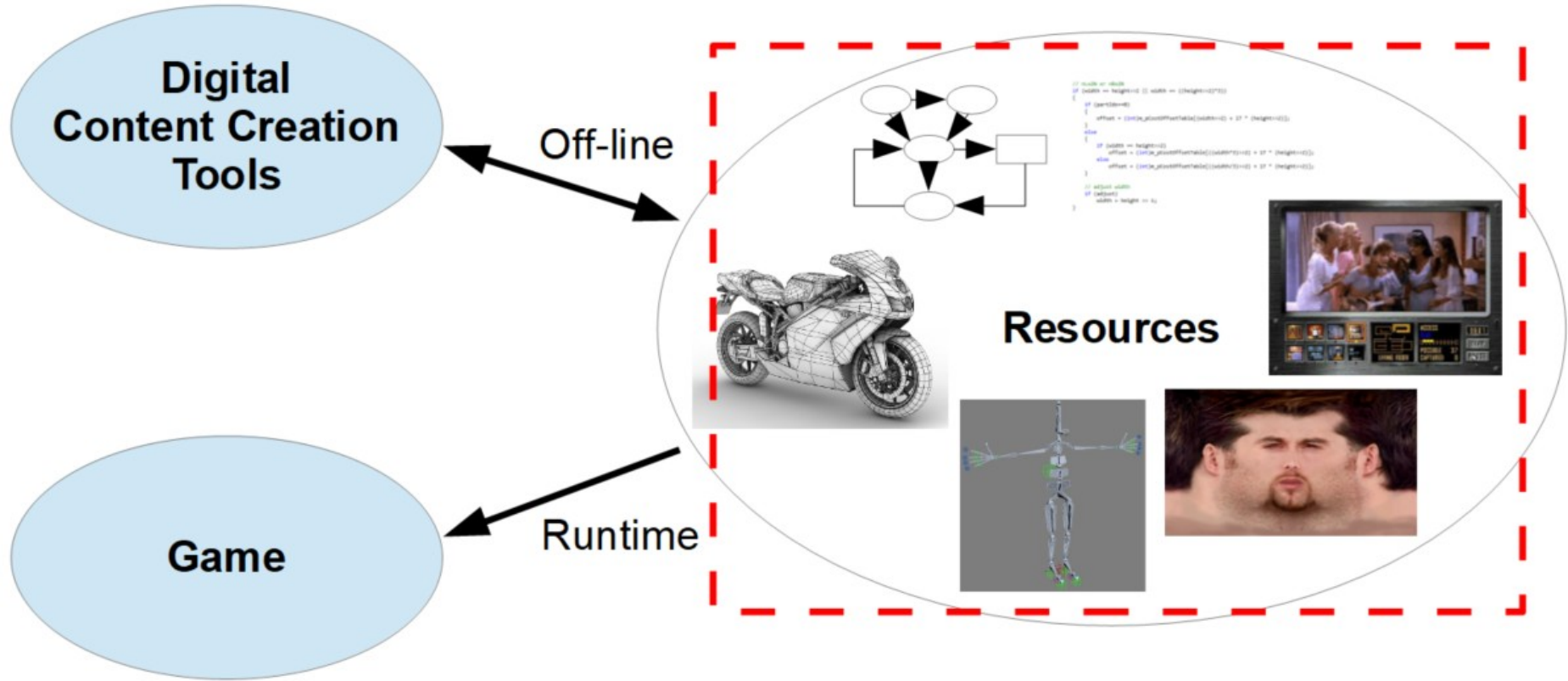


Image sources: <http://arsavar.com/wireframe-model#>, <http://maulik13.blogspot.de/2006/10/my-first-3d-character-rig.html>, http://images2.wikia.nocookie.net/half-life/en/images/a/ab/Trav_facemap.png, <http://tvtropes.org/pmwiki/pmwiki.php/Main/FullMotionVideo>

Game Resource Types



- 3D Object Meshes and Environments (geometry)
- 3D Mesh/Object Animation Data
- Map/Level Data
 - Trigger events, object types, scripts, ...
- Sprite and Texture Data
- Sound, Music, and Recorded Dialogue
- Video and Pre-rendered Cinematics

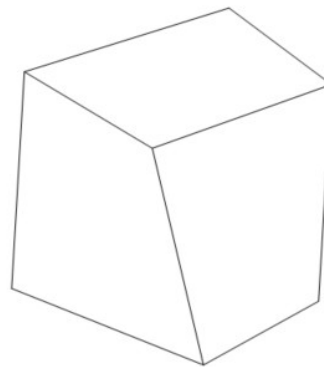
- Important aspects
 - Storage requirements and file size
 - File formats and scalability
 - Compression possibilities

3D Object Meshes and Environments



- A 3D object is a list of points in 3D space with accompanying data that describes how these points are organized into polygons and how the polygons should be rendered.

```
Vec3 AssymetricCubeVertices[8] =  
{  
    Vec3( 0.50f, 0.50f, -0.25f), // Vertex 0  
    Vec3(-0.50f, 0.50f, -0.25f), // Vertex 1  
    Vec3(-0.50f, 0.50f,  0.50f), // and so on  
    Vec3( 0.75f, 0.50f,  0.50f),  
    // ...  
    Vec3( 0.50f, -0.30f,  0.50f) // Vertex 7  
}  
  
Int AssymetricCubeIndices[12][3] =  
{  
    {0,1,2}, {0,2,3}, {0,4,5},  
    {0,5,4}, {1,5,6}, {0,1,6},  
    {2,6,7}, {2,7,3}, {3,7,4},  
    {3,4,0}, {4,7,6}, {4,6,5}  
}
```



Mesh storage requirements (1/2)



- Squashed Cube Example
 - 8 Vertices x 3 x sizeof(float) = 96 Bytes
 - 12 Triangles x 3 x sizeof(int) = 144 Bytes
 - Total: 240 Bytes
- More efficient ways to store triangle indices
 - Triangle Fans or Triangle Strips
 - Store n triangles with n + 2 indices instead of n x 3 indices

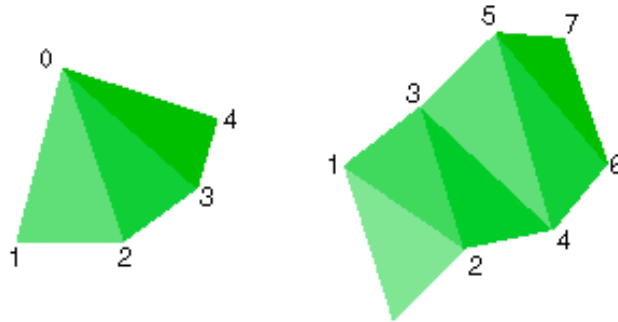
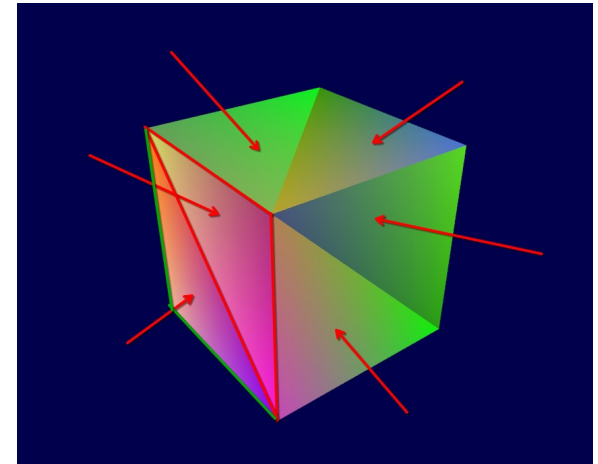


Image sources: https://techpubs.jurassic.nl/manuals/nt/developer/Optimizer_PG/sgi_html/ch03.html, <https://www.opengl-tutorial.org/beginners-tutorials/tutorial-4-a-colored-cube/>

Mesh storage requirements (2/2)



- Raw geometry data (2,000 vertices, 300 triangles, 100 triangle groups)

Object	Members	Size
Vertices	2,000 points @ 3 floats x 4 bytes each	24,000 bytes
Each triangle group	300 triangles @ 302 indices x 2 bytes each	604 bytes
All triangle group	100 groups @ 604 bytes	60,400 bytes
		84,400 bytes

- Storing simple versus complicated objects

Object	Members	Size
Simple textured and lit object (30 bytes / vertex)	302 indices per group x 100 groups @ 30 bytes	906,000 bytes
Complicated material info (80 bytes per vertex)	302 indices per group x 100 groups @ 80 bytes	2,416,000 bytes

=> Complicated objects are quite larger!

3D Mesh/Object Animation Data (1/2)



- Animations are stored as changes in position and orientation over time.
 - Position: $3 \times \text{float} = 12 \text{ bytes}$
 - Orientation:
 - Euler angles (roll, pitch yaw): $3 \times \text{float} = 12 \text{ bytes}$
 - Quaternions (X, Y, Z, W): $4 \times \text{float} = 16 \text{ bytes}$
 - Storing positions and orientations at fast time intervals
(e.g. 30 per second)
 - $12 \text{ bytes per position} + 12 \text{ bytes per orientation} = 24 \text{ bytes per sample}$
 - $30 \text{ samples per second} \times 24 \text{ bytes per sample} = 720 \text{ bytes per second}$
 - Characters are represented by a lot of discrete objects (movable bones)
 - $720 \text{ bytes per second} \times 30 \text{ bones} = 21,600 \text{ bytes per second}$

3D Mesh/Object Animation Data (2/2)



- Compression techniques
 - 15 samples per second are sufficient (most of the time)
 - Store changes in position and orientation when they happen
 - Objects typically don't move all the time
 - Not everything needs to be stored in every frame
 - Store full information every second or so (keyframes)
 - Jumping quickly to somewhere in the middle of the animation
 - Reduce small errors that can accumulate
 - Don't use floating point values
 - 2 bytes per angle => 65536 possible values => accuracy is 1/65536th of a circle
 - The Unreal Engine does just that!
- Rule of thumb: Size of in-game cinematics (without audio)
 - Assume the average of two characters moving simultaneously
 - Each cinematic averages 30 seconds
 - 25KB per character x 30 seconds = 1,53MB

Map/Level Data



- Mostly stored in a proprietary format
 - No standard format available
 - Matter of the programmer's preferences
- Types of data
 - Object information, AI scripts, dialogue lines, quest descriptions, ...
- Human readable format
 - Ini Files (key-value pairs)
 - XML (Extensible Markup Language)
 - Often converted to binary formats for the engine to load
 - Faster processing speed
 - Saves storage during runtime

Sprite and Texture Data



- 2 dimensional images (pixel data)
 - Applied to 3D models to imbue them with detail
 - Color textures (diffuse maps, etc.)
 - Special effects (normal maps, etc.)
 - User interface components like buttons, sliders and fonts
 - Particle images to create special effects like smoke or fire
 - Special purpose images like height-maps
- Other kinds include 1D or 3D textures, procedural textures, etc.
- Storage requirements depend on
 - Color Format
 - Compression
 - Runtime usage
- Textures need to fit into the texture memory!



Image sources: <https://blog.teamtreehouse.com/wp-content/uploads/2014/10/barrel1.gif>

Bitmap Color Formats (1/2)



- 32-bit (8888 RGBA)
 - Least compact / most information
- 24-bit (888 RGB)
 - Same as 8888, but no alpha channel
- 24-bit (565 RGB, A 8)
 - Nice looking with good alpha channel
 - The human eye is more sensitive to green than red or blue
- 16-bit (565 RGB)
 - Same as above, but no alpha channel
- 16-bit (555 RGB, A 1)
 - One bit for translucency
- 8-bit indexed
 - Uses an indexed table with colors and different levels of translucency

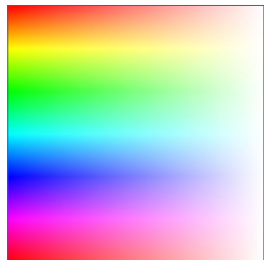


n-bits per pixel
RGBA-bits per color channel

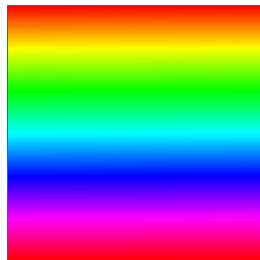
Bitmap Color Formats (2/2)



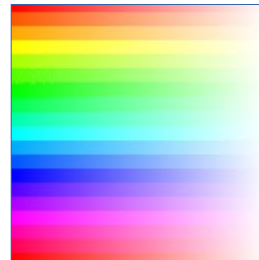
- Example images



8888 RGBA



555 RGB



Reduced Colours



Reduced Colours
1-bit translucency

- Trade-off between quality and file size
- Depends on the image content and on the intended usage
- We can't use the biggest format all the time!

Lossy Texture Compression



- Example images



High Quality



Low Quality

- Quality is perceptibly lost
- Difficult to find the optimal compression ratio
- Lossy => Lousy

Image sources: <https://www.elegantthemes.com/blog/tips-tricks/optimize-images-for-your-wordpress-website>

Mip-Maps



- A Mip-Map pre-calculates the image of a texture at different distances.
 - E.g.: full 128x128, has 64x64, 32x32, 16x16, 8x8, 4x4, 2x2, 1x1
 - The renderer will choose the proper Mip-Map levels to render
- A Full Mip-Map for a texture takes 33% more space
- Games almost always store their Mip-Maps in files and don't generate them on the fly.

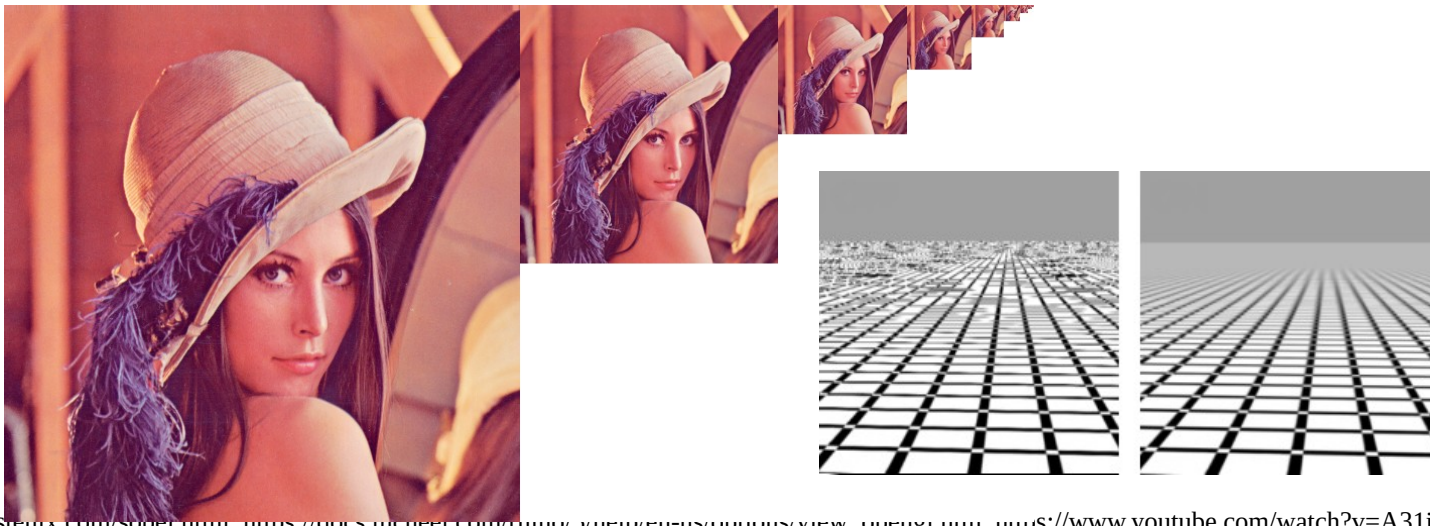


Image sources: <https://sistemx.com/sober.nam/>, https://docs.minecrafter.com/mmo/5/help/en-us/options/view_opengl.nam, https://www.youtube.com/watch?v=A31ifVF9_7c, <https://steamcommunity.com/sharedfiles/filedetails/?id=1209681886>,

Mip-Maps



- A Mip-Map pre-calculates the image of a texture at different distances
 - E.g.:
 - The mipmap level is chosen based on the distance from the viewer to the object.
- A Full Mip-Map chain contains all the mipmap levels from the highest to the lowest resolution.
- Games use mipmap chains to avoid aliasing artifacts like moiré patterns.



Image sources: <https://sistemx.com/sober.html>, https://docs.mtceer.com/mimo/3/help/en-us/options/view_opengl.html, https://www.youtube.com/watch?v=A31ifVF9_7c, <https://steamcommunity.com/sharedfiles/filedetails/?id=1209681886>,

Mip-M

- A Mip-
 - E.g
 - Th
- A Full
- Games



Image sources: https://system.com/robemany/https://document.com/minor/https://as/options/view_opengaming/https://www.mydatabase.com/watch?v=A31ifVF9_7c, <https://steamcommunity.com/sharedfiles/filedetails/?id=1209681886>,

Mip-M

- A Mip-
 - E.g.
 - Th
- A Full
- Games

tps: 142
in : 53
out: 63
loss: 0



m on the fly.

Image sources: https://system.com/robemulder/https://documentcloud.com/minor/bike/press_ad_options/view_opengraph/https://www.mydatabase.com/watch?v=A31ifVF9_7c, <https://steamcommunity.com/sharedfiles/filedetails/?id=1209681886>,

Sound, Music, and Recorded Dialogue (1/2)

- Sound formats in digital audio are
 - stored in mono or stereo
 - sampled at different frequencies
 - typically accurate to either 8 or 16 bits per sample
- Effects on audio quality and storage size are very obvious
 - Stereo needs twice the storage than mono
 - 8-bit sampled at 8KHz sounds like a telephone conversation
- The human ear is sensitive between 20Hz and 20,000Hz
 - Frequencies of WAV files are typically 44KHz, 22KHz, 11KHz
 - Nyquist–Shannon sampling theorem
 - $44\text{KHz} > 2 \times 20\text{KHz}$

Sound, Music, and Recorded Dialogue (2/2)

- Using different audio frequencies with digital formats

Format	Quality	Size per Second	Size per Minute
44.1KHz 16-bit stereo WAV	CD quality	172KB / second	10MB / minute
128Kbps stereo MP3	Near CD quality	17KB / second	1MB / minute
22.05KHz 16-bit stereo WAV	FM radio	86KB / second	5MB / minute
64kbps stereo MP3	FM radio	9KB / second	540KB / minute
11.025KHz 16-bit mono WAV	AM radio	43KB / second	2.5MB / minute
11.025KHz 8-bit mono WAV	Telephone	21KB / second	1.25MB / minute

- Audio can be compressed quite efficiently
 - MP3, Ogg, MPC, ...
 - Must be decompressed for playback
 - On demand (streaming) => needs CPU time
 - Entirely => needs space in memory

Video and Pre-rendered Cinematics



- **Approach 1:** Pre-rendered full-motion video
 - Playback of compressed video files
 - Large storage requirements
 - Can make use of real actors and scenery
 - Can hide loading screens
- **Approach 2:** In-game cutscenes
(using the game engine and renderer)
 - Exported animations and camera motion
 - Requires only relatively small animation files
 - Audio must be treated separately
 - ...which can make localization easier
 - Smoother transition from/to gameplay
 - Customizable characters work automatically
 - Graphics settings affect cutscene quality



Image sources: <http://www.thesixthaxis.com/2015/09/28/new-screens-show-off-the-witcher-3-hearts-of-stone-expansion/>, <http://www.pcgames.de/Wing-Commander-3-Heart-of-the-Tiger-Spiel-42565/News/Wing-Commander-3-Heart-of-the-Tiger-Unser-Retro-Video-stellt-euch-den-Sci-Fi-Klassiker-vor-847229/>

Resource Management - Overview

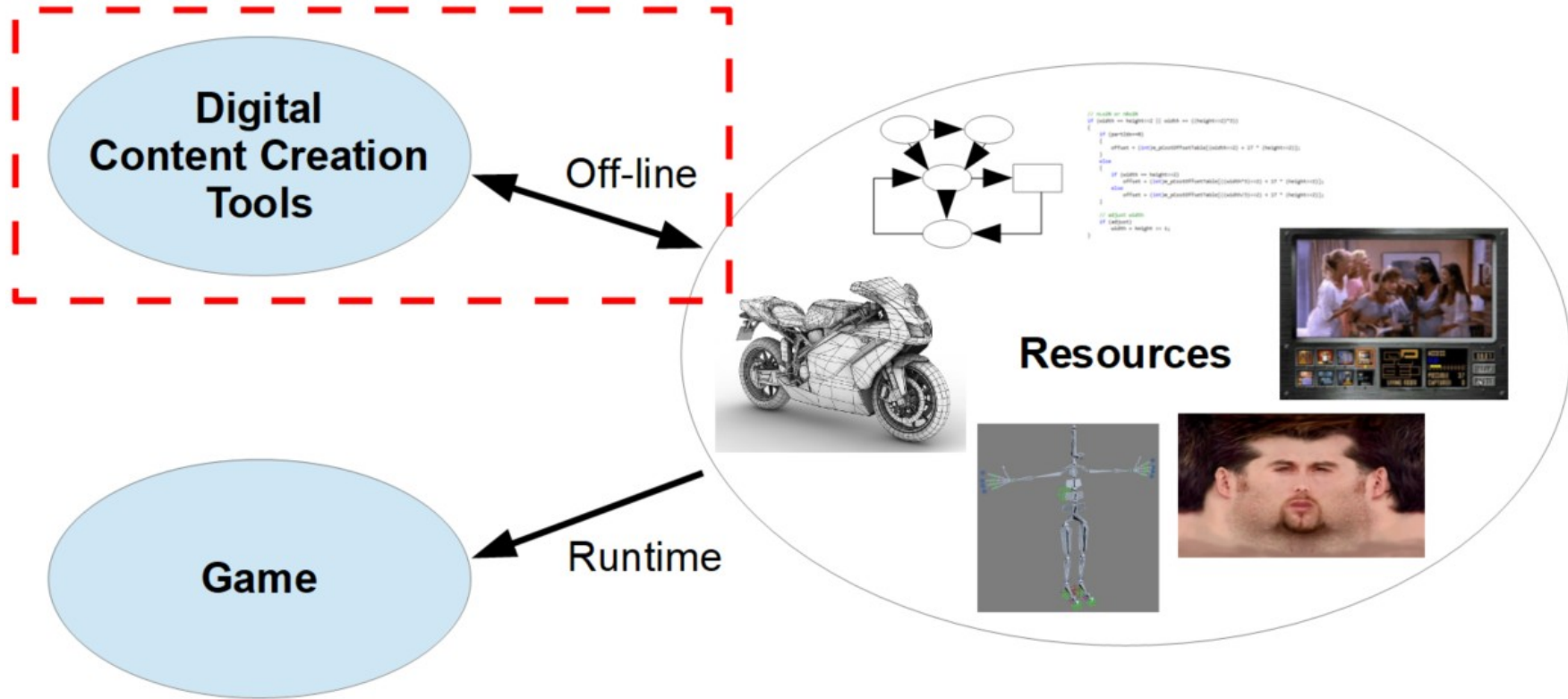


Image sources: <http://arsavar.com/wireframe-model#>, <http://maulik13.blogspot.de/2006/10/my-first-3d-character-rig.html>, http://images2.wikia.nocookie.net/half-life/en/images/a/ab/Trav_facemap.png, <http://tvtropes.org/pmwiki/pmwiki.php/Main/FullMotionVideo>

Revision control systems



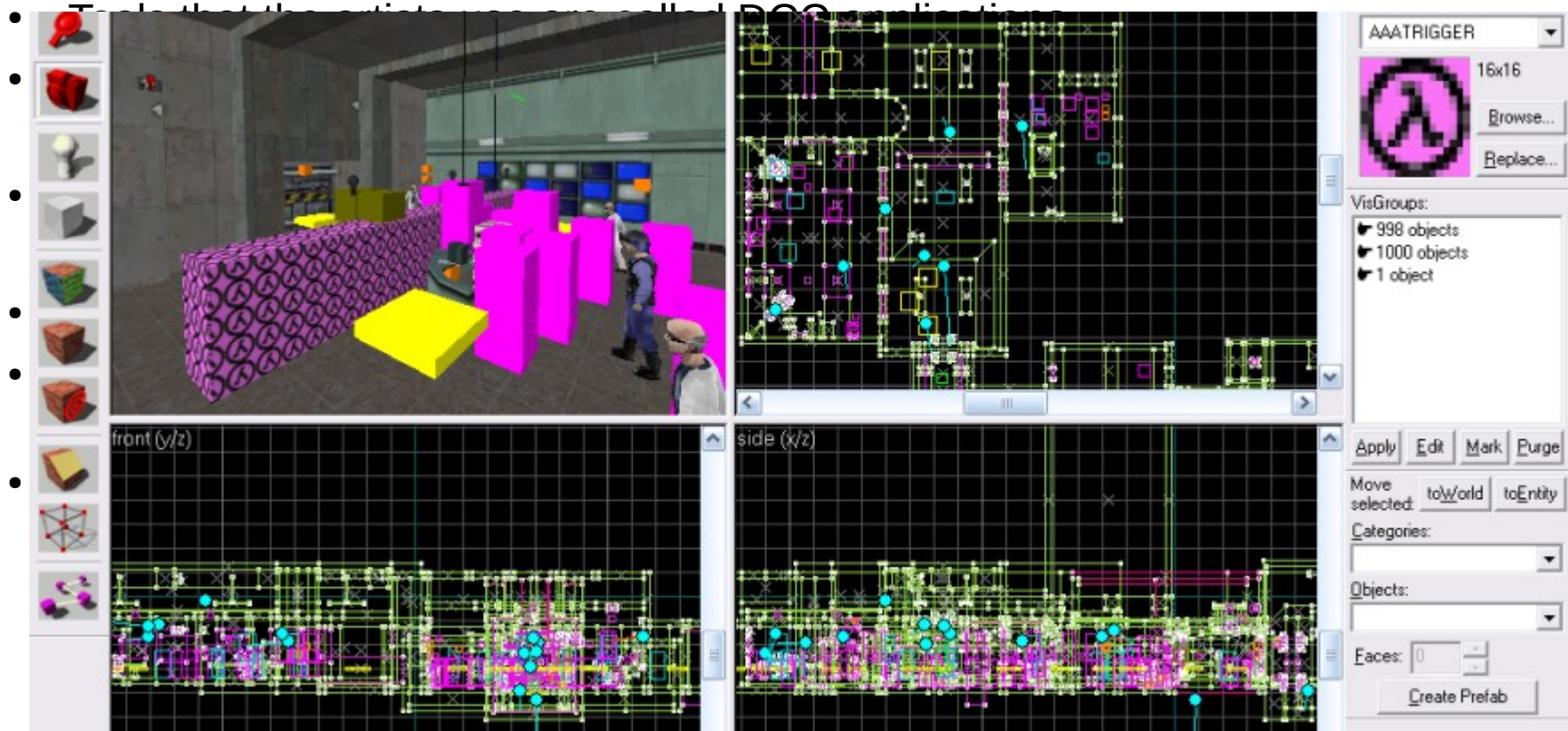
- Game assets are stored in a repository on a revision server
 - All versions of every file are available, from 1.0 to the latest
 - Multiple tags or branches may be created
- Clients work on local copies, called images
 - checkout an entire repository
 - update or commit working data
- Popular systems are Subversion (SVN), Concurrent Versions System (CVS), Perforce, AlienBrain, Git, Visual SourceSafe, ...
- Advantages
 - Secure storage, global access, central management
- Drawbacks
 - Amount of data, users may make errors

Digital Content Creation (DCC) Tools



- Tools that the artists use are called DCC applications
- Usually targeted at the creation of one particular type of data
 - Textures, particle effects, full motion video, ...
- Some tools can produce multiple data types (e.g. Maya, 3ds Max, Blender)
 - Models, skeletons, animations, ...
- Most engines provide a custom editor for laying out game worlds
- Some engines do make use of pre-existing tools for game world layout (e.g. Maya plug-ins)
- Tools must...
 - ...not be pretty (anything that gets the job done!)
 - ...be easy to use
 - ...be absolutely reliable

Digital Content Creation (DCC) Tools



Asset Conditioning Pipeline (1/2)



- Data formats used by DCC applications are rarely suitable for direct use in-game
 - The DCC app's in-memory model of the data is usually much more complex than what the game engine requires.
 - Additional data like history of edits, full hierarchy of 3D transformations, ...
 - The DCC application's file format is often too slow to read at run time, and in some cases it is a closed proprietary format.
- The data produced by a DCC application is usually exported to a more accessible standardized format, or a custom file format, for use in-game.
- Every game engine has an ACP of some sort

Image sources: -

Asset Conditioning Pipeline (2/2)



1) Exporters

- Either use custom plug-ins for the respective DCCs
- Or save the native (open) file format and use custom converter tools

2) Resource compilers

- “Massage” the raw data file in order to make it game-ready
 - E.g. rearrange a mesh's triangles to strips, compress texture bitmaps, calculate the arc lengths of a Catmull-Rom spline, and so on.
- Not all types resources need to be compiled

3) Resource linkers

- Multiple resource files sometimes need to be combined into a single useful package prior to being loaded by the game engine.
 - E.g. combine the data from multiple exported mesh files, multiple material files, a skeleton file, and multiple animation files into a single resource
- This mimics the process of linking together the object files of a compiled C++ program into an executable file (resource linking)

Image sources: https://www.researchgate.net/figure/An-example-of-a-Catmull-Rom-spline_fig1_50838845

Asset Conditioning Pipeline (2/2)



1) Exporters

- Either use
- Or save

2) Resource c

- "Messag"
 - E.g. a C
- Not all ty

3) Resource li

- Multiple being lo
 - E.g. mult
- This min

an executable file (resource linking)

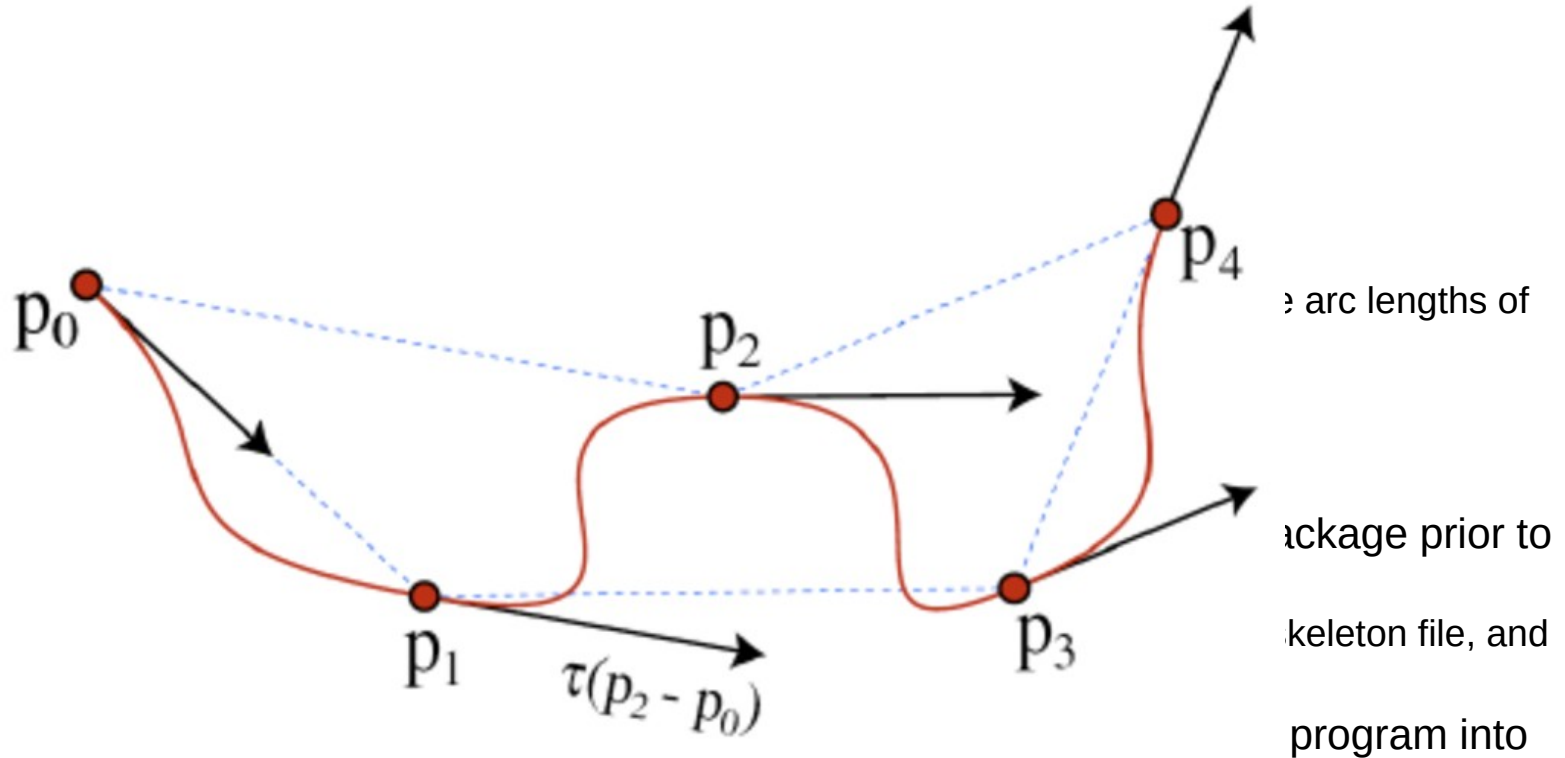


Image sources: https://www.researchgate.net/figure/An-example-of-a-Catmull-Rom-spline_fig1_50838845

Tools and the asset pipeline

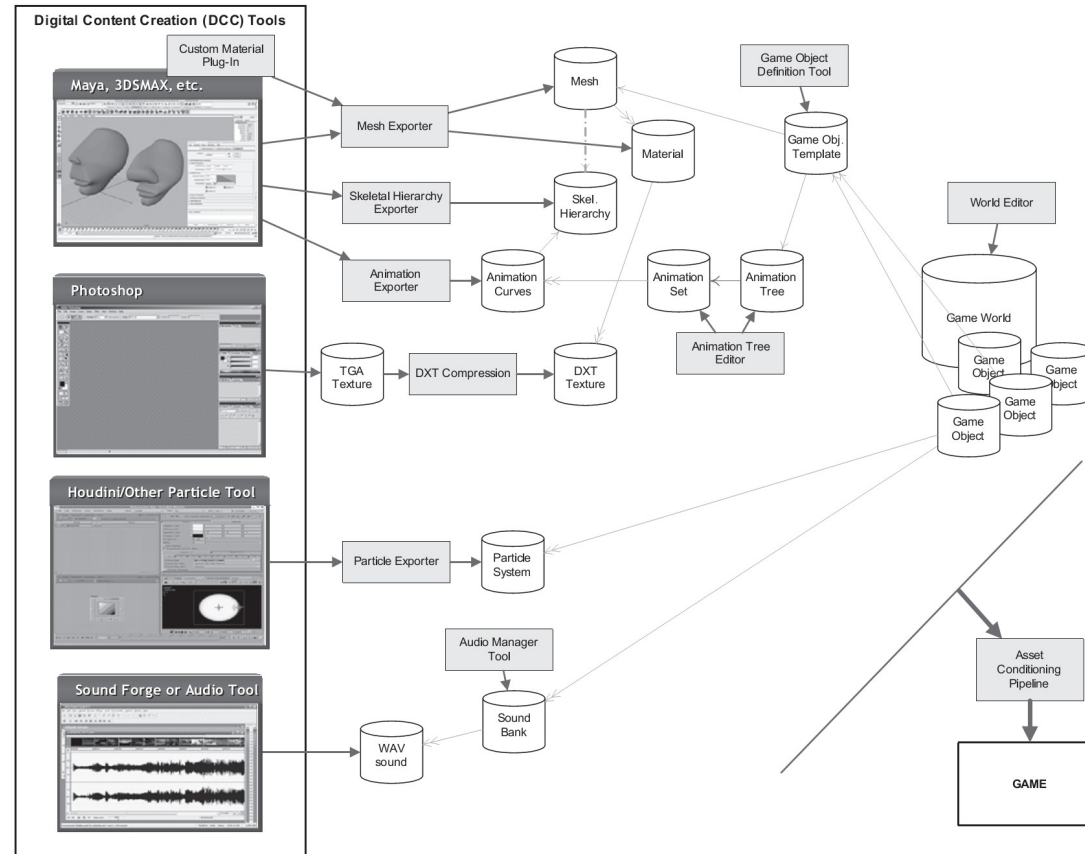


Image sources: Game Engine Architecture, 2nd edition, Taylor & Francis Ltd., 2014

Resource Database (1/2)



- For every resource that passes through the asset conditioning pipeline, there is some amount of metadata that describes how that resource should be processed.
 - E.g. which animation frames, which model represents which character, ...
- The resource database takes on vastly different forms in different game engines.
 - metadata might be embedded into the source assets themselves
 - each source resource file might be accompanied by a small text file
 - resource building metadata in a set of XML files
 - metadata in a true relational database

Image sources: -

Resource Database (2/2)



- A resource database must provide the following basic functionality:
 - The ability to deal with multiple types of resources.
 - The ability to create and delete resources.
 - The ability to inspect and modify existing resources.
 - The ability to move a resource's source file(s) from one location to another on-disk.
 - The ability of a resource to cross-reference other resources.
 - The ability to maintain referential integrity of all cross-references.
 - The ability to maintain a revision history, complete with a log of who made each change and why.
-and searching or querying is also very helpful.

Resource Database – Examples (1/2)



- Unreal Engine 4/5
 - Managed by their über-tool, the UnrealEd.
 - Responsible for literally everything, from resource metadata management to asset creation to level layout and more.
 - UnrealEd is a part of the game engine itself.
 - Assets to be created and then immediately viewed in their full glory, exactly as they will appear in-game.
 - The game can even be run from within UnrealEd in order to see if and how the assets work in-game.
 - The content browser allows a developer to access literally every resource that is consumed by the engine.
 - Less error prone than other engines, since checking asset validity is part of the production process.
 - Drawbacks
 - Resource data is stored in large package files (not good for Git, etc.).
 - Referential integrity could be better (uses dummy objects).

Resource Database – Examples (2/2)



- Naughty Dog's Uncharted / The Last of Us Engine
 - An XML file-based asset database, managed under Perforce.
 - A custom GUI tool was written to manage the contents of the database.
 - Many features for organization and management.
 - The asset conditioning pipeline consists of a set of resource exporters, compilers and linkers that are run from the command line.
 - Resource files are packaged as actors and levels.
 - Actor contains skeletons, meshes, materials, textures, animations.
 - Level contains static background meshes, materials and textures, and also level-layout information.
 - Drawbacks: Lack of visualization tools / tools are not fully integrated.
- OGRE's Resource Manager System
 - Very well-designed runtime resource manager.
 - A simple, consistent interface is used to load virtually any kind of resource.
 - Easy to include new resource managers.
 - Drawbacks
 - Runtime only solution (only provides some CMD exporters)
 - Metadata must be managed and applied manually.

Resource Management - Overview

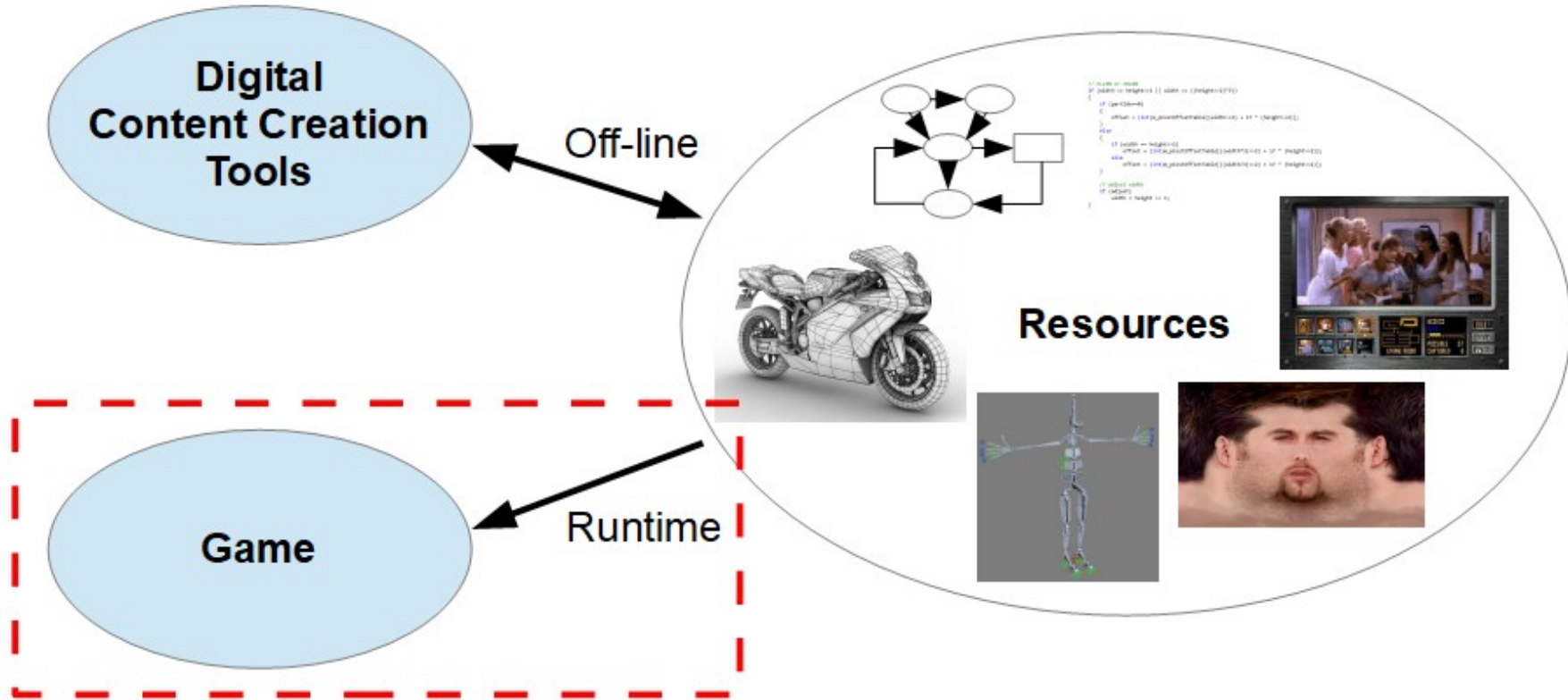
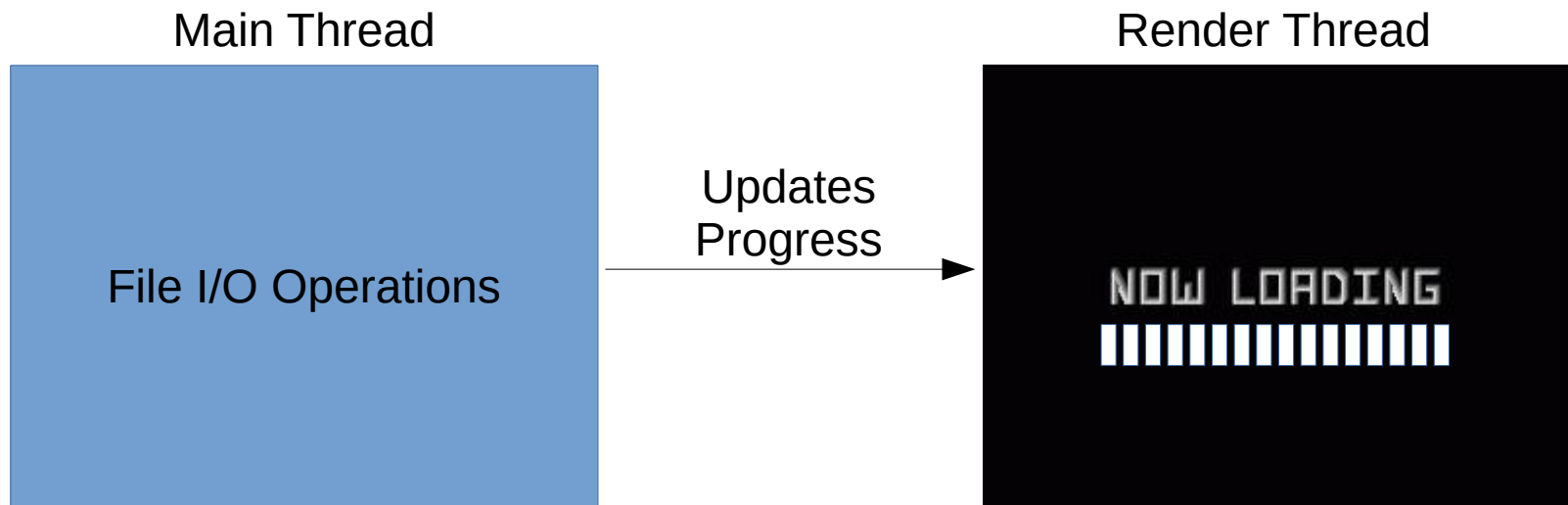


Image sources: <http://arsavar.com/wireframe-model#>, <http://maulik13.blogspot.de/2006/10/my-first-3d-character-rig.html>, http://images2.wikia.nocookie.net/half-life/en/images/a/ab/Trav_facemap.png, <http://tvtropes.org/pmwiki/pmwiki.php/Main/FullMotionVideo>

Synchronous File I/O



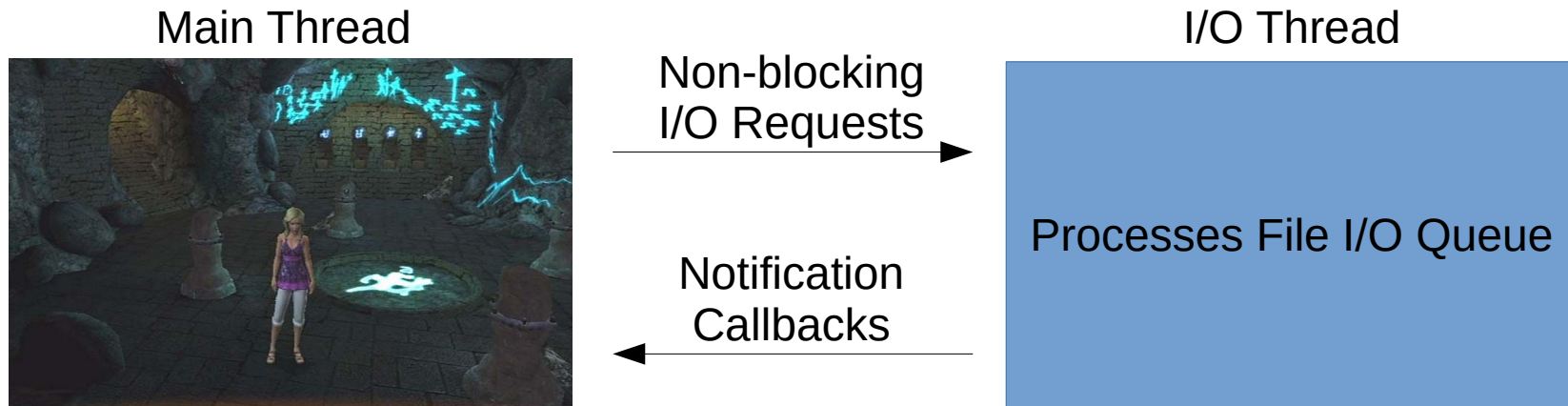
- The standard C file I/O libraries are synchronous
 - Caller must wait until the data has been completely transferred



Asynchronous File I/O (aka Resource Streaming)



- Program continues to run while its I/O requests are being satisfied



- Typical applications
 - Audio streaming (speech, music, atmo, ...)
 - Video streaming (during or in between gameplay)
 - On-demand loading of textures and geometry (pop-ups)
 - Pre-loading of upcoming content (next room or level)

Resource Files



- Game assets are typically packed together into resource files
- Example: Character data in a zip file
- Advantages
 - Saves storage capacity
 - File systems are organized in fixed-size blocks
 - Resource files can be compressed (e.g. ZIP)
 - Faster load times
 - Opening files is a slow operation
 - Fragmentation can cause multiple seeks
 - Security
 - Resource files can be encrypted
 - Hackers can't simply replace or alter single files
 - Logical organization
 - Everything that belongs together is in one file (level, chapter, cutscene, ...)

Character_0815.zip

description.xml, model.bin, skeleton.bin,
animations.bin, material.xml, texture.tga,
shader.cg, ...

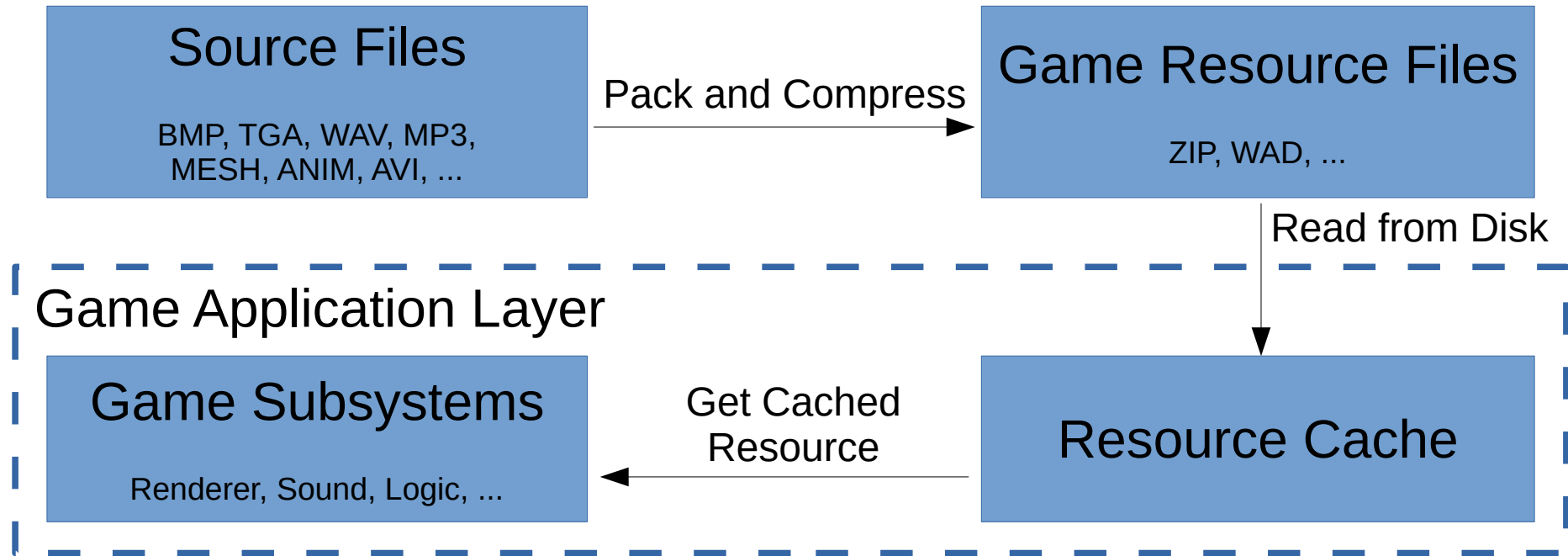


- Responsibilities
 - Ensures that only one copy of each unique resource exists in memory.
 - Globally Unique Identifiers (GUIDs)
 - Manages the lifetime of each resource.
 - Load-and-stay-resident (LSR) resources
 - Player character, fonts used on the heads-up display (HUD), ...
 - Some resources have a lifetime that matches that of a particular game level
 - Skybox, level collision geometry, certain enemies and NPCs, ...
 - Some resources have a lifetime that is shorter than the duration of the level
 - In-game cut-scene, speech samples, scripted events, ...
 - Some resources are streamed “live” as they play
 - Background music, ambient sound effects, or full-screen movies
 - Their lifetime is difficult to define because they never reside in memory completely
 - Maintains referential integrity.
 - Internal referential integrity (cross-references within a single resource)
 - External referential integrity (cross-references between resources)

The Resource Cache (1/3)



- We usually don't have enough memory to load all assets at once.
- We also don't want to wait while resources are streamed in.
- What we need is a resource cache!



The Resource Cache (2/4)

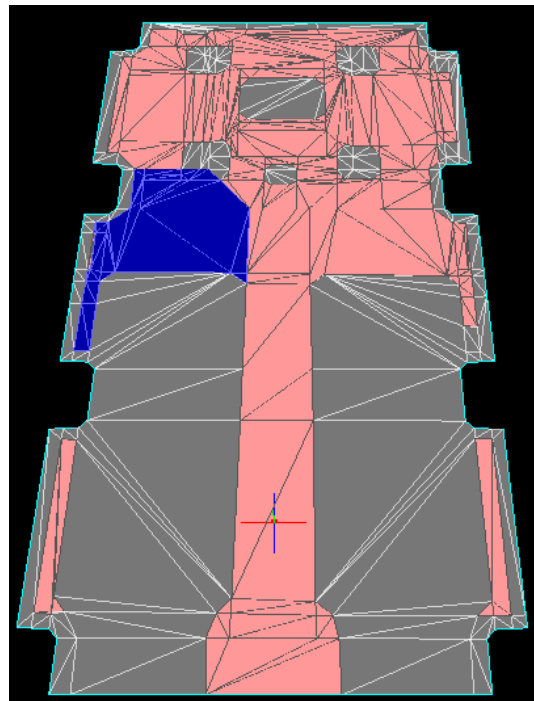


- Resource caches work on similar principles as any other cache
 - Things needed for the next step have probably been used already
- If the cache is full, we have to throw something out
- Three types of cache misses can occur
 - 1) Compulsory miss (request → not in cache → load into cache)
 - 2) Capacity miss (request → cache is full → throw something out → load)
 - 3) Conflict miss (data was previously thrown out, could have been avoided)
- Cache thrashing
 - Data needed for a single loop is bigger than the cache
 - Fatal for the game's frame-rate
 - Two options
 - Increase cache size (might not be possible)
 - Decrease memory requirements

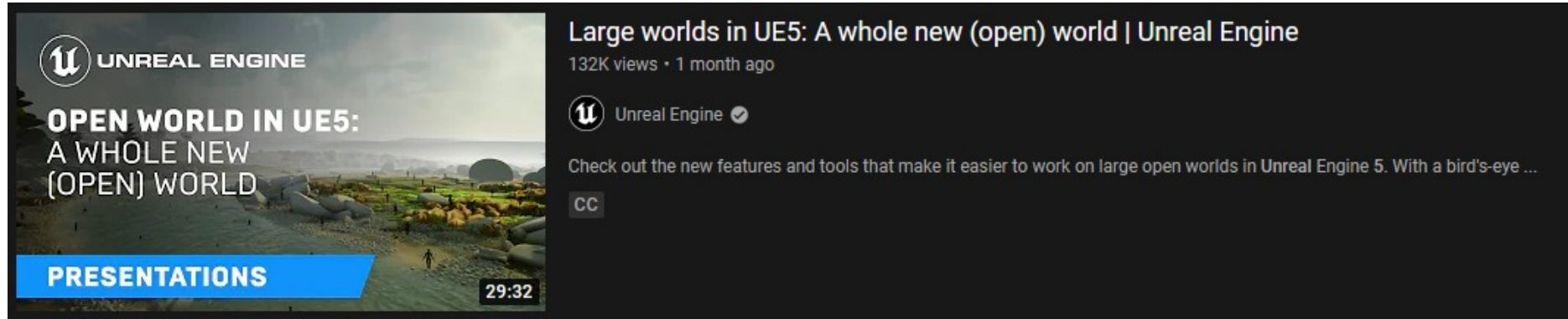
The Resource Cache (3/3)



- Predictive analysis of the game data
 - What will probably be needed next?
 - What can we throw out safely?
- Pinch points like portals, elevators, hallways, ...
- Open world games are more difficult to predict
- Predictive caching in *Der Fluch der Osterinsel*
 - Textures of adjacent rooms are streamed according to the avatar's current position.
 - Double-buffering of room textures



Large Worlds in Unreal 5



<https://www.youtube.com/watch?v=ZxJ5DG8Yto>
g